

NYCU DLP Lab5

1. Introduction

We focus on implementing MaskGIT for the inpainting task. During testing, images contain gray regions indicating missing information, which we aim to restore using MaskGIT.

The key implementation of this lab lies in three main areas: multi-head attention, transformer training, and inference inpainting.

The main goal is to compare the FID score with different settings of mask scheduling parameters and visualize the iterative decoding process for masks in latent domain or predicted images.

2. Implementation Details

A. The details of your model (Multi-Head Self-Attention)

Step:

- 1.Linear transformation for qkv vector
- 2.qkv dim transform to (B, n_h, n_t, d_p_h)
- 3.calculate the attention matrix
- 4.apply softmax (歸一化)
- 5.return to origin shape and do projection

```

def forward(self, x):
    ''' Hint: input x tensor shape is (batch_size, num_image_tokens, dim) '''
    batch_size, num_tokens, dim = x.shape

    # Linear transformation
    q = self.query(x) # q shape: (batch_size, num_tokens, dim)
    k = self.key(x)   # k shape: (batch_size, num_tokens, dim)
    v = self.value(x) # v shape: (batch_size, num_tokens, dim)
    # Reshape and transpose for multi-head attention
    # Reshape q, k, v to split dim into multiple heads and then transpose to get shape
    # (batch_size, num_heads, num_tokens, dim_per_head)
    q = q.view(batch_size, num_tokens, self.num_heads, dim // self.num_heads).permute(0, 2, 1, 3)
    k = k.view(batch_size, num_tokens, self.num_heads, dim // self.num_heads).permute(0, 2, 1, 3)
    v = v.view(batch_size, num_tokens, self.num_heads, dim // self.num_heads).permute(0, 2, 1, 3)

    # Compute attention matrix
    # Multiply q with transposed k and then scale by the square root of the key dimension size
    att = q @ k.permute(0, 1, 3, 2) # att shape: (batch_size, num_heads, num_tokens, num_tokens)
    #防止attention值太大
    att = att / np.sqrt(k.size(-1)) # Scaling to prevent extremely large values in softmax

    # Convert att to a tensor and apply softmax along the last dimension to get attention weights
    att = nn.functional.softmax(torch.tensor(att), dim=-1)
    att = self.dropout(att)

    # Multiply the attention weights with the value vectors to get the attention output
    y = att @ v # y shape: (batch_size, num_heads, num_tokens, dim_per_head)

    # Transpose y back to its original shape and combine the head outputs into a single dimension
    y = y.transpose(1, 2).contiguous().view(batch_size, num_tokens, dim)

    # Project the combined output to the original dimension using a linear layer
    return self.proj(y)

```

B. The details of your stage2 training (MVTM, forward, loss)

- TODO2: 1-1

```

##TODO2 step1-1: input x fed to vqgan encoder to get the latent and zq
@torch.no_grad()
def encode_to_z(self, x):
    zq, z_indices, _ = self.vqgan.encode(x) #_ is q_loss
    #raise Exception('TODO2 step1-1!')
    return zq, z_indices

```

- TODO2:1-2
 - implement different gamma function, including linear, cosine, square

```

##TODO2 step1-2:    decide mask schedule
def gamma_func(self, mode="cosine"):
    """Generates a mask rate by scheduling mask functions R.

    Given a ratio in [0, 1), we generate a masking ratio from (0, 1].
    During training, the input ratio is uniformly sampled;
    during inference, the input ratio is based on the step number divided by the total iteration number: t/T.
    Based on experiments, we find that masking more in training helps.

    ratio: The uniformly sampled ratio [0, 1) as input.
    Returns: The mask rate (float).
    一開始很高，後面越來越低
    """
    if mode == "linear":
        mask_rate=1-ratio
        #raise Exception('TODO2 step1-2!')
        #return None
    elif mode == "cosine":
        mask_rate=1-math.cos(math.pi * ratio / 2)
        #raise Exception('TODO2 step1-2!')
        #return None
    elif mode == "square":
        mask_rate=1-ratio**2
        #raise Exception('TODO2 step1-2!')
        #return None
    else:
        raise NotImplementedError
    return mask_rate

```

• TODO2:1-3

- Randomly create mask to train the transformer. The ratio is a random number
- return the predict result and ground truth

```

##TODO2 step1-3:
def forward(self, x, ratio): #X (B,C,H,W)
    ...
    z_indices:(B,num_tokens)
    logits : (B,num_tokens,num_codebook_vector+1)
    x : 圖
    ratio : 隨機sample的圖
    ...

    # encode 得codebook indice
    _,z_indices=self.encode_to_z(x) #ground truth [B*16*16]
    z_indices=z_indices.view(-1,self.num_image_tokens) #[B,256]
    # 隨機sample 一個mask出來
    mask = torch.bernoulli(torch.ones_like(z_indices) * ratio) #1代表被mask 0代表沒有
    mask_bool = mask.to(torch.bool)
    #套上mask
    input = mask_bool * self.mask_token_id + (~mask_bool) * z_indices
    #丟給transformer預測
    logits = self.transformer(input) # transformer predict the probability of tokens

    return logits, z_indices

```

• TODO2:1-4

- Use the CrossEntropyloss as the loss function

```

def train_one_epoch(self, train_loader, epoch):
    train_loss=0
    criterion = nn.CrossEntropyLoss()
    for img in (pbar := tqdm(train_loader, ncols=120)):
        img = img.to(args.device)
        #logits [B,256,1025] z_ind:[B,256]
        ratio = np.random.rand()
        logits,z_ind=self.model(img,ratio) #解碼出logits以及對應index

        #calculate loss
        #[B*256,1025] vs [B*256]
        #loss = criterion(logits.view(-1, logits.size(-1)), z_ind.view(-1))
        loss = criterion(logits.reshape(-1, logits.size(-1)), z_ind.reshape(-1))

        self.tqdm_bar(f'Train:{epoch}', pbar, loss.detach().cpu(), self.scheduler.get_last_lr()[0])
        # optimize
        self.optim.zero_grad()
        loss.backward()
        self.optim.step()

        train_loss+=loss.detach().cpu()

    train_loss/=len(train_loader)
    self.scheduler.step(loss)
    return train_loss

```

- The eval part:

```

@torch.no_grad()
def eval_one_epoch(self, val_loader, epoch):
    val_loss=0
    train_loss=[]
    for img in (pbar := tqdm(val_loader, ncols=120)):
        img = img.to(args.device)
        #logits [B,256,1025] z_ind:[B,256]
        ratio = np.random.rand()
        logits,z_ind=self.model(img,ratio) #解碼出logits以及對應index

        #calculate loss
        #[B*256,1025] vs [B*256]
        criterion = nn.CrossEntropyLoss()
        loss = criterion(logits.view(-1, logits.size(-1)), z_ind.view(-1))

        val_loss+=loss.detach().cpu()
        self.tqdm_bar(f'Val:{epoch}', pbar, loss.detach().cpu(), self.scheduler.get_last_lr()[0])
    val_loss/=len(val_loader)

    return val_loss

```

- Try different kind of optimizer and scheduler

```

def lr_lambda(self, steps):
    return min((steps+1)/args.warmup_steps, 1)
def configure_optimizers(self):
    ...
    optimizer = torch.optim.Adam(self.model.parameters(), lr=args.learning_rate, betas=(0.9, 0.96), weight_decay=1e-5)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
                                                            mode='min',
                                                            factor=0.1,
                                                            patience=2, # 2個epoch沒有降低 就調降
                                                            )
    ...

    optimizer = torch.optim.AdamW(self.model.parameters(), lr=args.learning_rate, betas=(0.9, 0.96))

    scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda=self.lr_lambda)
    #let the lr increase gradually
    return optimizer, scheduler

```

- TODO2:1-5
 - The detail of train code and save the train loss and val loss.

```

#TODO2 step1-5:
train_loss_list=[]
val_loss_list=[]
os.makedirs("img", exist_ok=True)
for epoch in range(args.start_from_epoch+1, args.epochs+1):
    ##### Training #####
    train_transformer.model.train()
    train_loss=train_transformer.train_one_epoch(train_loader,epoch)
    train_loss_list.append(train_loss.detach().cpu())
    ##### Val #####
    train_transformer.model.eval()
    with torch.no_grad():
        val_loss = train_transformer.eval_one_epoch(val_loader,epoch)
        val_loss_list.append(val_loss.detach().cpu())

    print(f'Epoch[{epoch}/{args.epochs}],Train_loss:{train_loss:.4f},Val_loss:{val_loss:.4f}')
    if epoch % args.save_per_epoch == 0:
        torch.save(train_transformer.model.transformer.state_dict(), f"./transformer_checkpoints/ckpt_{epoch}.pt")
        print(f"Model saved at epoch {epoch}")
    np.save(f"./img/train_loss.npy", np.array(train_loss_list))
    np.save(f"./img/val_loss.npy", np.array(val_loss_list))

```

C. The details of your inference for inpainting task (iterative decoding)

- TODO3:1-1
 - Apply the mask to `z_indices` based on the `mask`, and then pass the result into the transformer to get the output.
 - Apply softmax to the result and obtain the maximum value for each `token_value` to get the index of the maximum probability.
 - predicted probabilities add temperature annealing gumbel noise as confidence
 - If mask is False, the probability should be set to infinity, so that the tokens are not affected by the transformer's prediction
 - sort the confidence for the rank

- define how much the iteration remain predicted tokens by mask scheduling
- At the end of the decoding process, add back the original token values that were not masked to the predicted tokens.
- Return the `z_indices` and `mask_bc`, which will become the mask for the next iteration.

```
@torch.no_grad()
def inpainting(self, z_indices, ratio, mask, mask_num): # 上一次的預測tokens
    ...
    z_indices: [8,256]
    ratio: 控制要被mask的比例
    mask: 對應的遮罩
    ...

    # 取得被mask的地方
    ##### 處理 inpainting的原因 #####
    #套上mask
    input=mask*self.mask_token_id+(~mask)*z_indices
    #丟給Transformer
    logits = self.transformer(input)#[8,num_tokens,num_codeBlockVector+1]
    #Apply softmax to convert logits into a probability distribution across the last dimension.
    logits = torch.nn.functional.softmax(logits, dim=-1)

    #FIND MAX probability for each token value
    z_indices_predict_prob, z_indices_predict = torch.max(logits, dim=-1) #value index (8,256),(8,256) #最大值即codebook對應的label

    ##### 處理被 mask圖 修復進度 需要input到下一個 mask_b #####
    #predicted probabilities add temperature annealing gumbel noise as confidence
    #Gumbel 噪聲來增加隨機性
    #溫度參數控制隨機性，低溫度使預測更確定，高溫度增加隨機性
    g = torch.distributions.Gumbel(0, 1).sample(sample_shape=z_indices_predict_prob.shape).to(z_indices_predict_prob.device) # gumbel noise
    temperature = self.choice_temperature * (1 - ratio)
    confidence = z_indices_predict_prob + temperature * g
    #hint: If mask is False, the probability should be set to infinity, so that the tokens are not affected by the transformer's prediction
    #sort the confidence for the rank
    #define how much the iteration remain predicted tokens by mask scheduling
    #At the end of the decoding process, add back the original token values that were not masked to the predicted tokens

    #若mask是0 則取無限大，只想關注mask=1的部分
    confidence = torch.where(mask == 0, torch.tensor(float('inf')).to(confidence.device), confidence) #(1,256)
    #sort the confidece for rank
    n = math.ceil(mask_num * self.gamma(ratio)) #算有幾個1
    values,indices=torch.topk(confidence,n,largest=False) #由小排到大將mask=1 predict的結果
    max_values=values[0,-1] #取出這些最小值的最大值
    mask_bc=confidence<=max_values #若比<=max_values 設true 代表要繼續predice , > 則設成false 不用predict

    #At the end of the decoding process, add back the original token values that were not masked to the predicted tokens
    z_indices_predict = mask * (z_indices_predict) + (~mask) * z_indices #把1的改成z_indices_preict
    return z_indices_predict, mask_bc
```

- TODO3:1-2
 - calculate the mask_num and pass it to inpainting
 - update the ratio of total_iter

```

self.model.eval()
with torch.no_grad():
    _, z_indices = self.model.encode_to_z(image) #z_indices: masked tokens (b,16*16)
    mask_num = mask_b.sum().item()
    z_indices = z_indices.view(-1, 256) #變成(b,256)

    #z_indices_predict=z_indices
    mask_bc=mask_b #上一階段mask的value
    mask_b=mask_b.to(device=self.device)
    mask_bc=mask_bc.to(device=self.device)

    #raise Exception('TODO3 step1-1!')
    ratio = 0
    #iterative decoding for loop design
    #Hint: it's better to save original mask and the updated mask by scheduling separately
    for step in range(self.total_iter):
        if step == self.sweet_spot:
            break
        ratio = step / self.total_iter #this should be updated

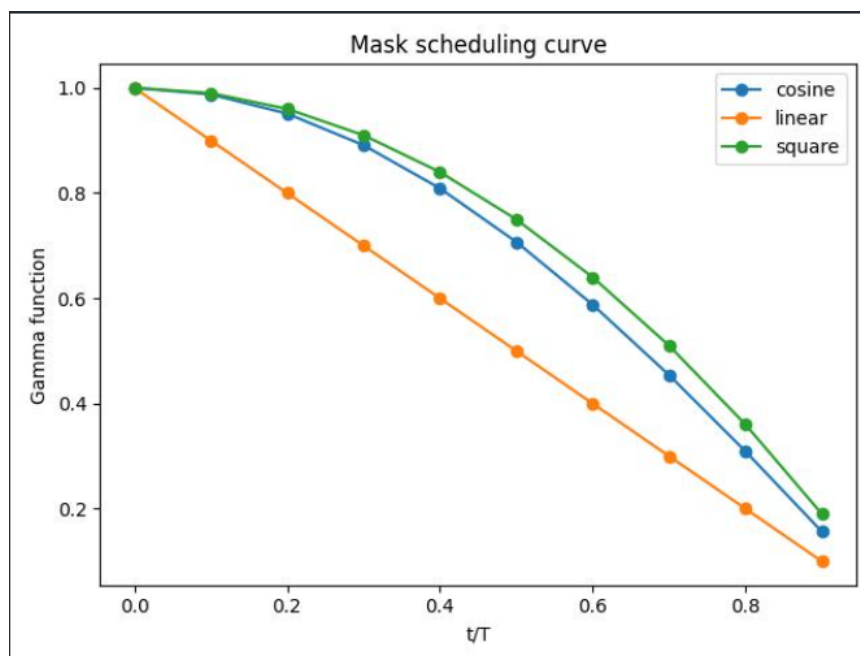
    z_indices_predict, mask_bc = self.model.inpainting(z_indices, ratio, mask_bc, mask_num=mask_num)

```

3. Discussion

A. Mask Scheduling curve

- Visualize the Gamma function result



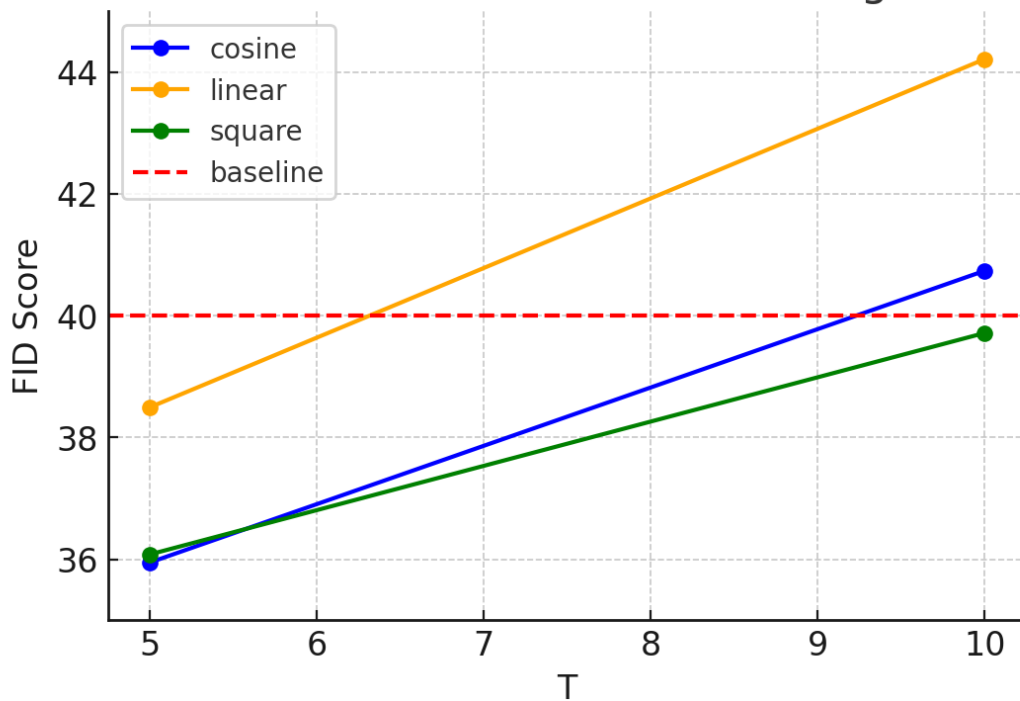
B. Try more epoch to train the model:

- The result seems to be worse, I guess that model may overfitting.

C. The different mask scheduling graph

- The result under different T and mask scheduling method
 - All scheduling methods perform relatively well at $T=5$. As T increases, the FID scores generally rise, indicating a decline in performance.
 - The linear method perform the worst.

FID Score with Different Mask Scheduling Parameters



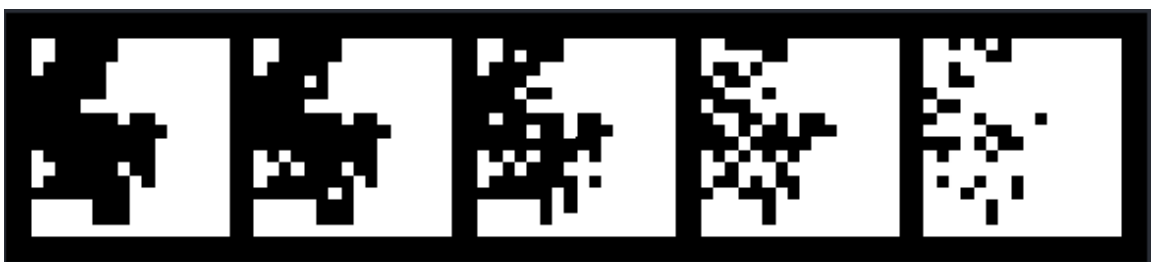
Experiment Results

Part1: Prove your code implementation is correct

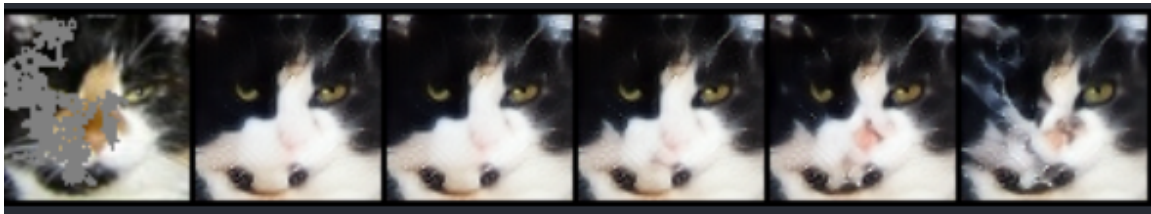
1.Show iterative decoding

1. Cosine:

a. mask in latent domain



b. Predicted image

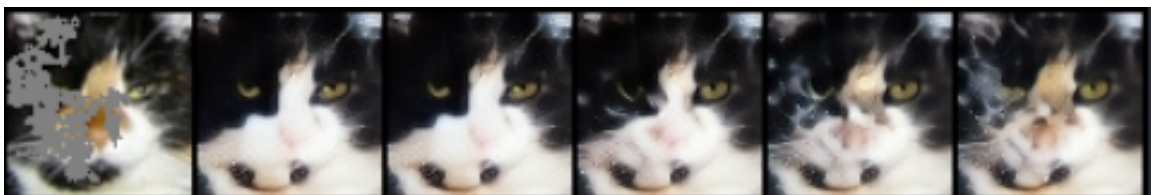


2. Linear:

a. mask in latent domain



1. Predicted image



3. Square:

a. mask in latent domain



b. Predicted image

